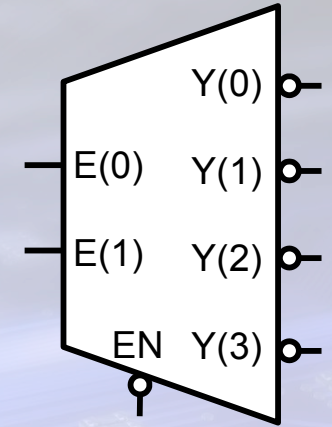


11. Problemas

1. Describese en VHDL un **decodificador binario de 2 a 4** con entrada de habilitación, **EN**. La entrada de habilitación y las salidas son activas a nivel lógico bajo. Las salidas del decodificador se declararán como un **std_logic_vector**. De esta forma se simplifica la descripción del decodificador
2. Describese en VHDL un **comparador de magnitud de n bits** con entrada de habilitación **EN**. Tendrá tres salidas **A_mayor_B**, **A_igual_B** y **A_menor_B** para indicar el resultado de la comparación. La entrada de habilitación y las salidas son activas a nivel lógico alto
3. A la señal Y, que es del tipo de datos std_logic, se le asignan al mismo tiempo los valores: '1', 'L', 'H' y 'Z'. ¿Qué valor se le asignará finalmente a la señal?

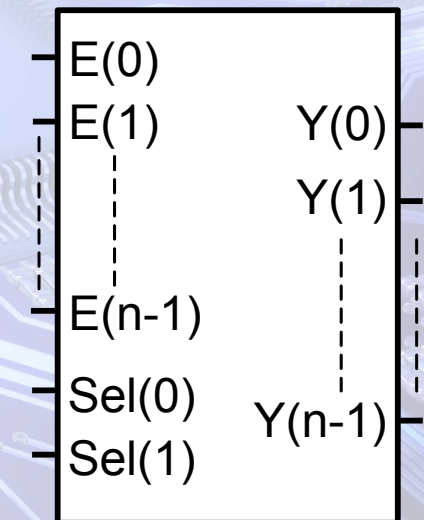
Indíquese el proceso de resolución



11. Problemas

4. Describese en VHDL un comparador de magnitud de 3 números (A, B y C) de n bits. Tendrá dos salidas, Mayor y Menor. Mayor se pondrá a 1, si $A > B > C$, y Menor se pondrá a 1, si $A < B < C$.
5. Describese en VHDL el desplazador de barril de la figura. La salida Y es igual a la entrada E desplazada a la izquierda tantos bits como indique la entrada Sel. Siempre se realiza la operación de desplazamiento lógico a la izquierda. Téngase en cuenta que el desplazador de barril es un circuito combinacional y que no se puede usar los operadores de desplazamiento analizados en el tema 3, ya que no están definidos para el tipo de datos `std_logic_vector` en el package `std_logic_1164`.

Desplazador de barril de n bits



11. Problemas

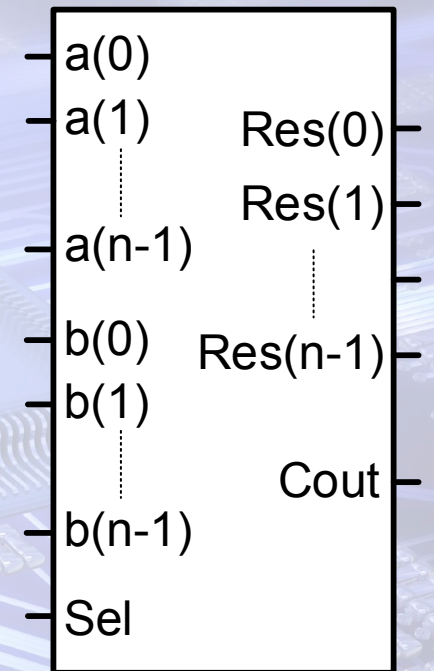
6. Sumador binario de n bits

- Describir en VHDL el sumador binario de n bits de la figura. **A** y **B** son las entradas de n bits en las que conecta los operandos a sumar. **Sum** es el resultado de la suma de n bits de A y B, y **Cout** la salida de acarreo. Los puertos A, B y Sum se declararán del tipo `std_logic_vector`, y Cout del tipo `std_logic`
- **Dos opciones:**
 - ✓ **Diseño no estandarizado**, aunque válido para la mayoría de herramientas
 - Se usa el package `std_logic_unsigned`
 - Se declaran los port del tipo `std_logic_vector`
 - ✓ **Diseño estandarizado**
 - Se usa el package `numeric_std`
 - Se declaran los port del tipo `std_logic_vector`
 - Se convierte a **unsigned** los operandos de entrada al sumarlos
 - Se convierte a `std_logic_vector` el resultado al asignarlos al port de salida
 - Como los tipos `std_logic_vector` y `unsigned` están estrechamente relacionados se usa la **conversión explícita de datos**

11. Problemas

7. Sumador-restador binario de n bits

- **A** y **B** son las entradas de n bits en las que conecta los operandos de entrada.
- **Sel** es la entrada de control que determina la operación a realizar, de forma que si está a 0 se selecciona la suma de A y B.
- **Res** es la salida de n bits en la que se representa el resultado de la operación indicada por la entrada Sel.
- **Cout** es la salida de acarreo (suma) o de préstamo (resta).
- Realícese la descripción teniendo en cuenta que el sumador-restador debe poder operar tanto con números binarios sin signo como con signo representados en el sistema de representación en C2.
- Se determinará si falta alguna señal de entrada y/o de salida en la representación a nivel de bloque de la figura, y de ser así, incluirlas en el diseño.



11. Problemas

❖ Sumador-restador binario de n bits

■ Modos de funcionamiento:

Sel = 0 $\Rightarrow$ Res = A + B

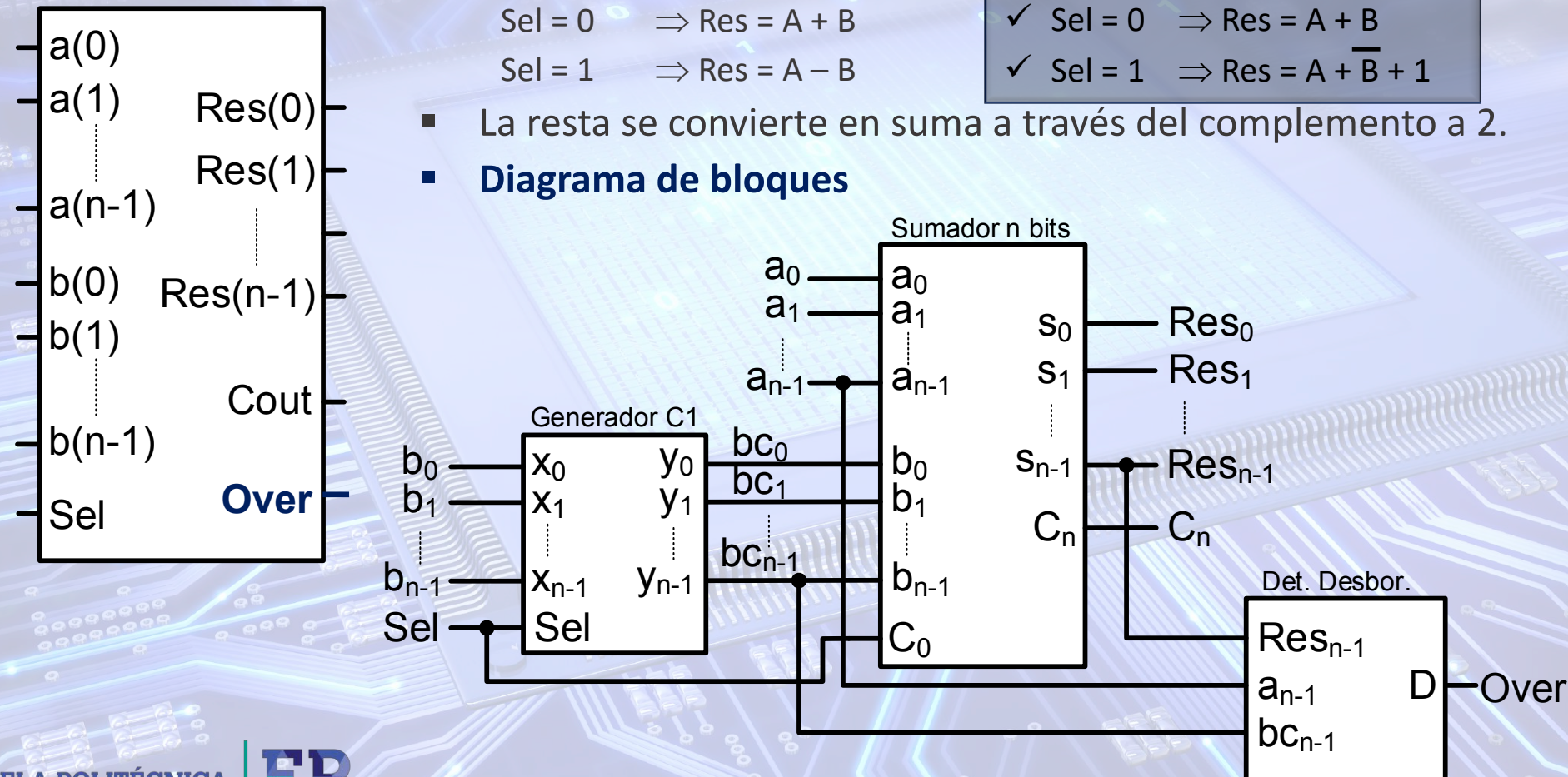
Sel = 1 $\Rightarrow$ Res = A - B

✓ Sel = 0 $\Rightarrow$ Res = A + B

✓ Sel = 1 $\Rightarrow$ Res = A + $\overline{B}$ + 1

■ La resta se convierte en suma a través del complemento a 2.

■ Diagrama de bloques



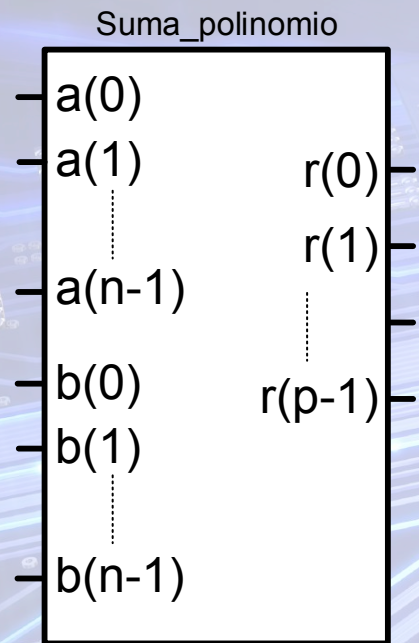
11. Problemas

8. Cálculo de un polinomio

- **A** y **B** son las entradas de n bits de los operandos de entrada.
- **R** es la salida de p bits en la que se representa el resultado de la operación aritmética **$A \cdot B + 3A + 5B$** , sin que se produzca desbordamiento, y teniendo en cuenta que $n \geq 3$.

■ Aclaraciones:

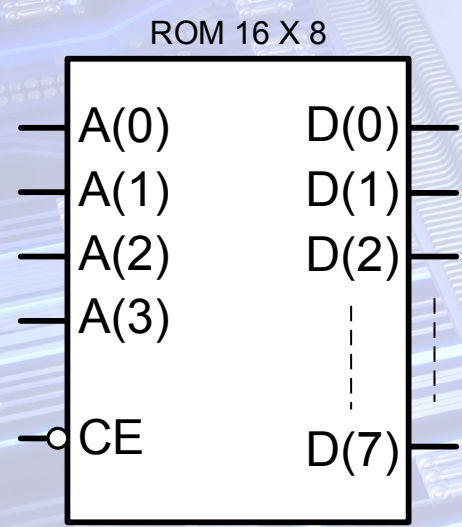
- ✓ Si A es de n_1 bits y B de n_2 bits, el producto de A por B , tiene n_1 más n_2 bits, es decir, $2 \cdot n$ bits.
- ✓ Considerando $A = B$, y sacando factor común a A :
 - $R = A \cdot (A+8)$
 - Dado que la suma de $A + 8$, puede generar un acarreo, el resultado de esa suma será de $n+1$ bits.
- ✓ Por tanto, el número de bits de la salida (p) será:
 - $P = n + (n + 1) = 2 \cdot n + 1$.



11. Problemas

9. Diseño de una ROM 16x8

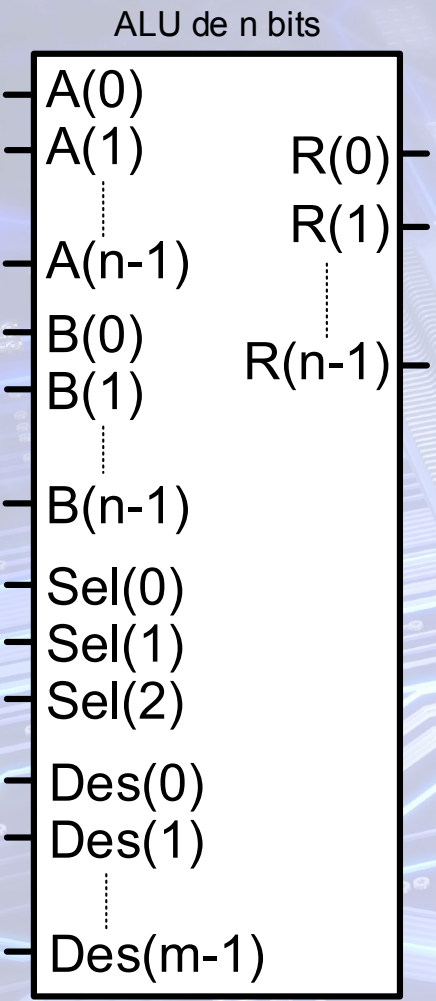
- Tiene 16 palabras de 8 bits cada una
 - ✓ Tendrá 4 patillas de direcciones (A) y 8 patillas de datos (D)
- La entrada CE es el Chip Enable o habilitación de chip
 - ✓ Es **activa a nivel lógico bajo** de forma que sólo se puede leer una palabra de la ROM si CE está activa.
 - ✓ Si está desactiva las patillas de datos estarán en alta impedancia.



Dirección (H)	Contenido	Dirección (H)	Contenido
0	0	8	87 ₁₆
1	3 ₁₆	9	44 ₁₆
2	45 ₁₆	A	B9 ₁₆
3	EF ₁₆	B	AA ₁₆
4	55 ₁₆	C	FF ₁₆
5	2C ₁₆	D	58 ₁₆
6	79 ₁₆	E	9D ₁₆
7	AD ₁₆	F	66 ₁₆

11. Problemas

10. Diseño 5. ALU de n bits



- **Por defecto** se considerará que es de **4 bits**
- La descripción debe ser **completamente estándar con el IEEE**
- Tiene que **realizar las operaciones** indicadas en la tabla, según el valor de las **entradas de selección de operación Sel**
- Para las **operaciones de desplazamiento** no se podrá usar el operador de concatenación.
 - ✓ Se usarán los operadores de desplazamiento lógicos correspondientes declarados en el **Package Numeric_std**
 - ✓ Se realizará el **número de desplazamientos** indicado por la entrada **Des**. Por defecto, se considerará, **m = 2**

Sel	Operación	Sel	Operación
000	AND lógica de A y B	100	A + 1
001	XOR lógica de A y B	101	A - 1
010	A + B	110	Desp. Lógico a la derecha de A
011	A - B	111	Desp. Lógico a la izquierda de A